



Name: SINA RAHINI

Project Timeline: 6/7/2025-10/10/2025

Introduction

The purpose of this project is to show technical skills and problem solving ability by working through something similar to a real NBA data science project. **Note:**

Throughout this document, any **season** column represents the year each season started. For example, the 2015-16 season will be in the dataset as 2015. We may refer to a season by just this number (e.g. 2015) instead of the full text (e.g. 2015-16).

Answers

Part 1

Question 1: 26 4-in-6 stretches in OKC's draft schedule.

Question 2: 25.1 4-in-6 stretches on average.

Question 3:

- Most 4-in-6 stretches on average: CHA (28.1)
- Fewest 4-in-6 stretches on average: NYK (22.2)

Question 4: This is a written question. Please leave your response in the document under Question 4.

Question 5:

- BKN Defensive eFG%: 54.3%
- When opponent on a B2B: 53.5%

Part 2

Please show your work in the document, you don't need anything here.

Part 3

Question 9:

- Most Helped by Schedule: MIL (74.2 wins)
- Most Hurt by Schedule: DET (-96.3 wins)



Setup and Data

```
In [ ]: # %pip install numpy pandas matplotlib scipy jupyter notebook openpyxl requests
```

```
In [4]: import pandas as pd
import os
# Note, you will likely have to change these paths. If your data is in the same
# the paths will likely be fixed for you by deleting ../../Data/schedule_project
user = os.getlogin()
schedule = pd.read_csv(f"C:/Users/{user}/Python Projects/Data/schedule.csv")
draft_schedule = pd.read_csv(f"C:/Users/{user}/Python Projects/Data/schedule_24_
locations = pd.read_csv(f"C:/Users/{user}/Python Projects/Data/locations.csv")
game_data = pd.read_csv(f"C:/Users/{user}/Python Projects/Data/team_game_data.cs
```

```
In [5]: # Quick look at schedule dataset.
print("First few rows of schedule dataset:")
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
display(schedule.head())
print("schedule variable types:")
print(schedule.dtypes)
print(schedule.info)
```

First few rows of schedule dataset:

	season	gamedate	team	opponent	home	win
0	2014	2015-04-15	SAC	LAL	0	1
1	2014	2015-04-15	NYK	DET	1	0
2	2014	2015-04-15	DEN	GSW	0	0
3	2014	2015-04-15	MIA	PHI	0	1
4	2014	2015-04-15	UTA	HOU	0	0

schedule variable types:

```
season      int64
gamedate    object
team        object
opponent    object
home        int64
win         int64
```

dtype: object

```
<bound method DataFrame.info of
in
0      2014  2015-04-15  SAC      LAL      0      1
1      2014  2015-04-15  NYK      DET      1      0
2      2014  2015-04-15  DEN      GSW      0      0
3      2014  2015-04-15  MIA      PHI      0      1
4      2014  2015-04-15  UTA      HOU      0      0
...      ...      ...      ...      ...      ...      ...
23953   2023  2023-10-25  CHI      OKC      1      0
23954   2023  2023-10-24  PHX      GSW      0      1
23955   2023  2023-10-24  GSW      PHX      1      0
23956   2023  2023-10-24  DEN      LAL      1      1
23957   2023  2023-10-24  LAL      DEN      0      0
```

[23958 rows x 6 columns]>

```
In [6]: # Quick look at draft_schedule dataset.
print("First few rows of draft_schedule dataset:")
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
```



```
display(draft_schedule.head())
print("draft_schedule variable types:")
print(draft_schedule.dtypes)
print(draft_schedule.info)
```

First few rows of draft_schedule dataset:

	season	gamedate	team	opponent	home	win
0	2024	2025-04-13	DEN	HOU	0	1
1	2024	2025-04-13	OKC	NOP	0	1
2	2024	2025-04-11	OKC	UTA	0	1
3	2024	2025-04-11	DEN	MEM	1	1
4	2024	2025-04-09	OKC	PHX	0	1

draft_schedule variable types:

```
season      int64
gamedate    object
team        object
opponent    object
home        int64
win         int64
```

dtype: object

```
<bound method DataFrame.info of
0      2024  2025-04-13  DEN    HOU    0    1
1      2024  2025-04-13  OKC    NOP    0    1
2      2024  2025-04-11  OKC    UTA    0    1
3      2024  2025-04-11  DEN    MEM    1    1
4      2024  2025-04-09  OKC    PHX    0    1
..      ...      ...    ...    ...    ...    ...
155     2024  2024-10-27  OKC    ATL    1    1
156     2024  2024-10-26  OKC    CHI    0    1
157     2024  2024-10-26  DEN    LAC    1    0
158     2024  2024-10-24  DEN    OKC    1    0
159     2024  2024-10-24  OKC    DEN    0    1
```

[160 rows x 6 columns]>

```
In [7]: # Quick look at Locations dataset.
print("First few rows of locations dataset:")
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
display(locations.head())
print("locations variable types:")
print(locations.dtypes)
print(locations.info)
```

First few rows of locations dataset:

	team	latitude	longitude	timezone
0	ATL	33.757233	-84.396340	Eastern
1	BOS	42.366310	-71.062226	Eastern
2	BKN	40.682657	-73.975282	Eastern
3	CHA	35.225186	-80.839336	Eastern
4	CHI	41.880629	-87.674048	Central



locations variable types:

team object
latitude float64
longitude float64
timezone object
dtype: object

```
<bound method DataFrame.info of
0 ATL 33.757233 -84.396340 Eastern
1 BOS 42.366310 -71.062226 Eastern
2 BKN 40.682657 -73.975282 Eastern
3 CHA 35.225186 -80.839336 Eastern
4 CHI 41.880629 -87.674048 Central
5 CLE 41.496577 -81.688076 Eastern
6 DAL 32.790314 -96.810148 Central
7 DEN 39.748550 -105.007711 Mountain
8 DET 42.341949 -83.054694 Eastern
9 GSW 37.767927 -122.387326 Pacific
10 HOU 29.750907 -95.362061 Central
11 IND 39.763990 -86.155566 Eastern
12 LAC 34.043006 -118.267360 Pacific
13 LAL 34.043006 -118.267360 Pacific
14 MEM 35.138213 -90.050449 Central
15 MIA 25.781349 -80.187441 Eastern
16 MIL 43.043625 -87.916838 Central
17 MIN 44.979455 -93.276190 Central
18 NOP 29.949003 -90.081904 Central
19 NYK 40.750562 -73.993471 Eastern
20 OKC 35.463536 -97.515298 Central
21 ORL 28.538954 -81.384276 Eastern
22 PHI 39.901180 -75.171972 Eastern
23 PHX 33.445830 -112.071206 Mountain
24 POR 45.531497 -122.666783 Pacific
25 SAC 38.580207 -121.499350 Pacific
26 SAS 29.427002 -98.437482 Central
27 TOR 43.643521 -79.379082 Eastern
28 UTA 40.768262 -111.901091 Mountain
29 WAS 38.898006 -77.021002 Eastern>
```

```
In [8]: # Quick look at game_data dataset.
print("First few rows of game_data dataset:")
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
display(game_data.head())
print("game_data variable types:")
print(game_data.dtypes)
print(game_data.shape)
```

First few rows of game_data dataset:

	season	gametype	nbageameid	gamedate	offensiveteamid	off_team_name	off_te
0	2016	2	21600495	2016-12-30	1610612740	New Orleans Pelicans	N
1	2016	2	21600495	2016-12-30	1610612752	New York Knicks	N
2	2021	2	22100943	2022-03-03	1610612742	Dallas Mavericks	D
3	2021	2	22100943	2022-03-03	1610612744	Golden State Warriors	G
4	2016	2	21601032	2017-03-18	1610612741	Chicago Bulls	C



```
game_data variable types:
season                int64
gametype              int64
nbgameid              int64
gamedate              object
offensivenbateamid    int64
off_team_name         object
off_team              object
off_home              int64
off_win               int64
defensivenbateamid    int64
def_team_name         object
def_team              object
def_home              int64
def_win               int64
fg2made               int64
fg2missed             int64
fg2attempted          int64
fg3made               int64
fg3missed             int64
fg3attempted          int64
fgmade                int64
fgmissed              int64
fgattempted           int64
ftmade                int64
ftmissed              int64
ftattempted           int64
reboffensive          int64
rebdefensive          int64
reboundchance         int64
assists               int64
stealsagainst         int64
turnovers             int64
blocksagainst         int64
defensivefouls        int64
offensivefouls        int64
shootingfoulsdrawn    int64
possessions           int64
points                int64
shotattempts          int64
andones               int64
shotattemptpoints     int64
dtype: object
(23958, 41)
```

Part 1 -- Schedule Analysis

In this section, you're going to work to answer questions using NBA scheduling data.

Question 1

QUESTION: How many times are the Thunder scheduled to play 4 games in 6 nights in the provided 80-game draft of the 2024-25 season schedule? (Note: clarification, the stretches can overlap, the question is really "How many games are the 4th game played over the past 6 nights?")

```
In [9]: import numpy as np

# Filter for OKC games
okc_games = draft_schedule[draft_schedule['team'] == 'OKC'].copy()
okc_games['gamedate'] = pd.to_datetime(okc_games['gamedate'])
okc_games = okc_games.sort_values('gamedate').reset_index(drop=True)
```



```
# Initialize counter for 4-in-6 stretches.
# This counts the number of games where OKC plays their 4th game in a 6-day window
count_4_in_6 = 0
dates = okc_games['gamedate']

# For each game, check if there are 3 other OKC games in the previous 5 days (6-day window)
for i in range(len(dates)):
    window_start = dates[i] - pd.Timedelta(days=5)
    games_in_window = ((dates >= window_start) & (dates <= dates[i])).sum()
    if games_in_window >= 4:
        count_4_in_6 += 1

print(f"OKC has {count_4_in_6} 4-in-6 stretches in the draft schedule.")
```

OKC has 26 4-in-6 stretches in the draft schedule.

ANSWER 1:

26 4-in-6 stretches in OKC's draft schedule.

Question 2

QUESTION: From 2014-15 to 2023-24, what is the average number of 4-in-6 stretches for a team in a season? Adjust each team/season to per-82 games before taking your final average.

```
In [10]: # Ensure gamedate is datetime
schedule['gamedate'] = pd.to_datetime(schedule['gamedate'])

# Function to count 4-in-6 stretches for a team/season
def count_4_in_6_for_team(df):
    df = df.sort_values('gamedate').reset_index(drop=True)
    dates = df['gamedate']
    count = 0
    for i in range(len(dates)):
        window_start = dates[i] - pd.Timedelta(days=5)
        games_in_window = ((dates >= window_start) & (dates <= dates[i])).sum()
        if games_in_window >= 4:
            count += 1
    return count

# Group by team and season, count stretches and adjust to per-82 games
results = []
for (team, season), group in schedule.groupby(['team', 'season']):
    n_games = len(group)
    stretches = count_4_in_6_for_team(group)
    # Adjust to per-82 games
    stretches_per_82 = stretches * 82 / n_games if n_games > 0 else 0
    results.append({'team': team, 'season': season, 'stretches_per_82': stretches_per_82})

results_df = pd.DataFrame(results)

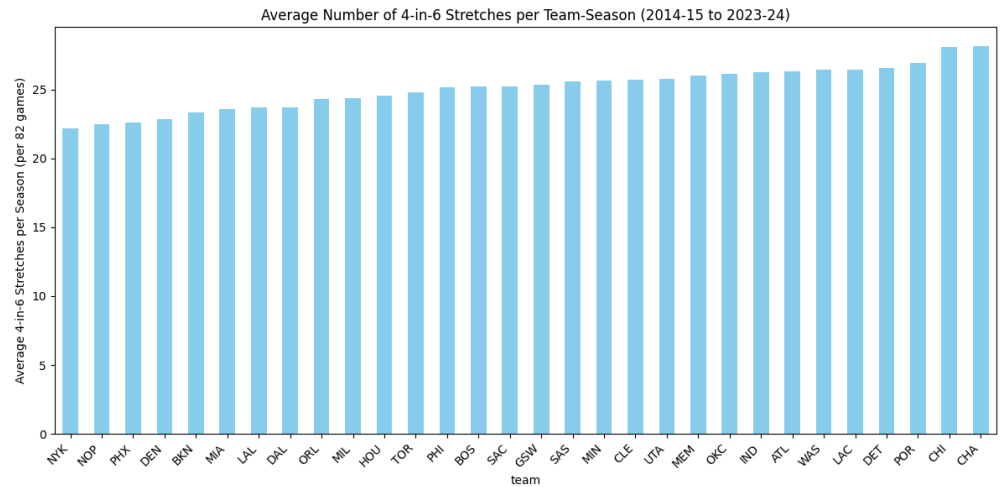
# Average across all team-seasons
average_4_in_6 = results_df['stretches_per_82'].mean()
print(f"Average number of 4-in-6 stretches per team-season (per 82 games): {average_4_in_6}")
```

Average number of 4-in-6 stretches per team-season (per 82 games): 25.1

```
In [11]: import matplotlib.pyplot as plt
# Calculate average 4-in-6 stretches per team (across all seasons)
team_avg = results_df.groupby('team')['stretches_per_82'].mean()
plt.figure(figsize=(12,6))
team_avg_sorted = team_avg.sort_values()
team_avg_sorted.plot(kind='bar', color='skyblue')
```



```
plt.ylabel('Average 4-in-6 Stretches per Season (per 82 games)')
plt.title('Average Number of 4-in-6 Stretches per Team-Season (2014-15 to 2023-24)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



ANSWER 2:

25.1 4-in-6 stretches on average.

Question 3

QUESTION: Which of the 30 NBA teams has had the highest average number of 4-in-6 stretches between 2014-15 and 2023-24? Which team has had the lowest average? Adjust each team/season to per-82 games.

```
In [12]: # Find team with highest and lowest average 4-in-6 stretches per season (per 82
most_stretches_team = team_avg.idxmax()
most_stretches_value = team_avg.max()
fewest_stretches_team = team_avg.idxmin()
fewest_stretches_value = team_avg.min()

# Printing both the team names and their average values
print(f"Most 4-in-6 stretches on average: {most_stretches_team} ({most_stretches_value})")
print(f"Fewest 4-in-6 stretches on average: {fewest_stretches_team} ({fewest_stretches_value})")
```

Most 4-in-6 stretches on average: CHA (28.1)
 Fewest 4-in-6 stretches on average: NYK (22.2)

ANSWER 3:

- Most 4-in-6 stretches on average: CHA (28.1)
- Fewest 4-in-6 stretches on average: NYK (22.2)

Question 4

QUESTION: Is the difference between most and least from Q3 surprising, or do you expect that size difference is likely to be the result of chance?

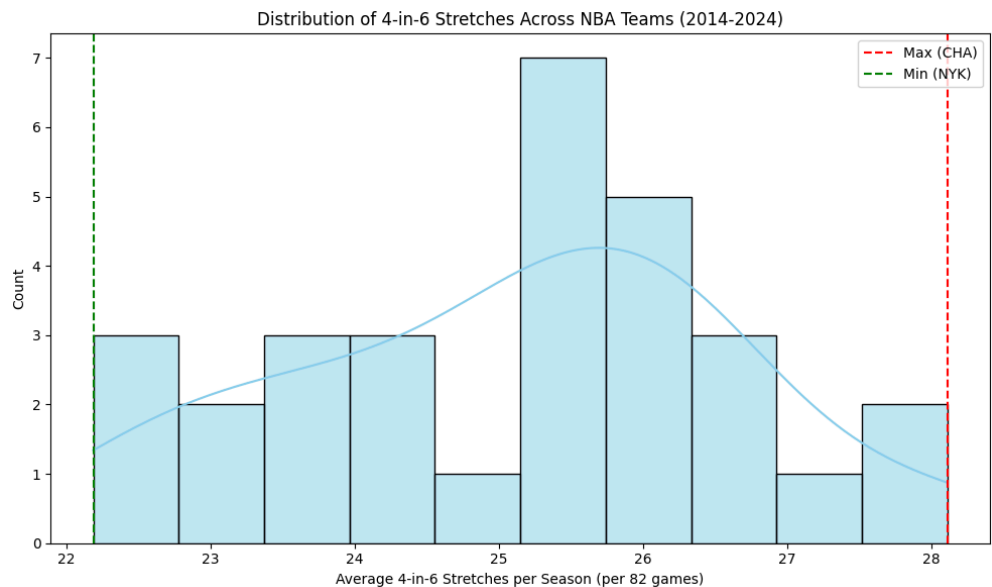
```
In [13]: import seaborn as sns

import scipy.stats as stats
import matplotlib.pyplot as plt
```



```
# Prepare data
values = team_avg.values
labels = team_avg.index

# Plot distribution of team averages
plt.figure(figsize=(10,6))
sns.histplot(values, bins=10, kde=True, color='skyblue')
plt.axvline(most_stretches_value, color='red', linestyle='--', label=f'Max ({most_stretches_team})')
plt.axvline(fewest_stretches_value, color='green', linestyle='--', label=f'Min ({fewest_stretches_team})')
plt.xlabel('Average 4-in-6 Stretches per Season (per 82 games)')
plt.title('Distribution of 4-in-6 Stretches Across NBA Teams (2014-2024)')
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [14]: # Z-score for max and min teams
z_max = (most_stretches_value - values.mean()) / values.std()
z_min = (fewest_stretches_value - values.mean()) / values.std()
print(f"Z-score for most (CHA): {z_max:.2f}")
print(f"Z-score for least (NYK): {z_min:.2f}")

# Test for normality
stat, p_normal = stats.shapiro(values)
print(f"Shapiro-Wilk test for normality: p-value = {p_normal:.3f}")

# Outlier test (Grubbs' test for max and min)
def grubbs_test(values, extreme):
    N = len(values)
    mean = values.mean()
    std = values.std()
    G = abs(extreme - mean) / std
    # Critical value for alpha=0.05
    t = stats.t.ppf(1-0.05/(2*N), N-2)
    G_crit = ((N-1)/np.sqrt(N)) * np.sqrt(t**2/(N-2+t**2))
    return G, G_crit

G_max, G_crit_max = grubbs_test(values, most_stretches_value)
G_min, G_crit_min = grubbs_test(values, fewest_stretches_value)
print(f"Grubbs' test statistic for max: {G_max:.2f} (critical: {G_crit_max:.2f})")
print(f"Grubbs' test statistic for min: {G_min:.2f} (critical: {G_crit_min:.2f})")

# Boxplot for visual outlier detection
plt.figure(figsize=(8,2))
sns.boxplot(x=values, color='lightgray')
plt.scatter([most_stretches_value, fewest_stretches_value], [0,0], color='red')
```




```
plt.title('Boxplot of Team 4-in-6 Stretch Averages')
plt.xlabel('Average 4-in-6 Stretches per Season (per 82 games)')
plt.show()
```

Z-score for most (CHA): 1.96

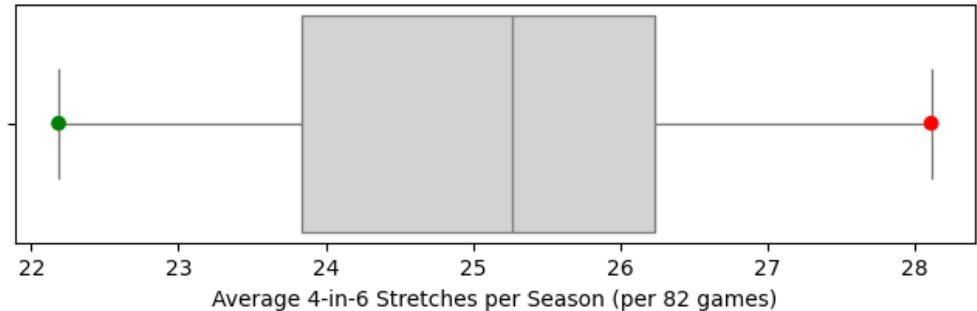
Z-score for least (NYK): -1.90

Shapiro-Wilk test for normality: p-value = 0.514

Grubbs' test statistic for max: 1.96 (critical: 2.91)

Grubbs' test statistic for min: 1.90 (critical: 2.91)

Boxplot of Team 4-in-6 Stretch Averages



ANSWER 4:

- Instead of answering based off the eye test (which seemed like a very reasonable range between the most and the least), I went ahead and ran some testing for "normality." The results showed the following, in summary:
 - According to the Shapiro-Wilk test for normality, checking the z-scores for the most and least team averages of 4-in-6 stretches, both sides fall just under 2 standard deviations away from the overall average. This gives us a p-value of 0.514, so since that is far greater than our critical alpha value of 0.05 (or even 0.10 if we wanted to be more generous), there's no evidence to suggest that the range between the highest and the lowest average is out of the ordinary or surprising at all.

Question 5

QUESTION: What was BKN's defensive eFG% in the 2023-24 season? What was their defensive eFG% that season in situations where their opponent was on the second night of back-to-back?

```
In [15]: # Filter for games where BKN is the defensive team in the 2023-24 season
bkn_def = game_data[(game_data['def_team'] == 'BKN') & (game_data['season'] == 2023)]

# Calculate eFG% (eFG% equation was defined using my own research/prior knowledge)
efg = (bkn_def['fgmade'].sum() + 0.5 * bkn_def['fg3made'].sum()) / bkn_def['fgat'].sum()
print(f"BKN Defensive eFG% in 2023-24: {efg * 100:.1f}%")
```

BKN Defensive eFG% in 2023-24: 54.3%

```
In [16]: # Get all games in 2023 for each team
game_data['gamedate'] = pd.to_datetime(game_data['gamedate'])
games_2023 = game_data[game_data['season'] == 2023].copy()

# For each BKN defensive game, check if off_team played the previous day
bkn_def['gamedate'] = pd.to_datetime(bkn_def['gamedate'])
b2b_mask = []
for idx, row in bkn_def.iterrows():
    off_team = row['off_team']
    game_date = row['gamedate']
    # Find previous game for off_team before this date
```



```
team_games = games_2023[(games_2023['off_team'] == off_team) | (games_2023['home_team'] == off_team)]
team_games = team_games[team_games['gamedate'] < game_date]
if not team_games.empty:
    prev_game_date = team_games['gamedate'].max()
    # If previous game was exactly one day before, it's a B2B
    b2b_mask.append((game_date - prev_game_date).days == 1)
else:
    b2b_mask.append(False)

bkn_def['opponent_b2b'] = b2b_mask

# Filter for games where opponent was on a B2B
bkn_b2b_games = bkn_def[bkn_def['opponent_b2b']]

# Calculate eFG% for these games
efg_b2b = (bkn_b2b_games['fgmade'].sum() + 0.5 * bkn_b2b_games['fg3made'].sum()) / (bkn_b2b_games['fgattempts'].sum() + 0.5 * bkn_b2b_games['fg3attempts'].sum())
print(f"BKN Defensive eFG% when opponent on B2B in 2023-24: {efg_b2b * 100:.1f}%")
```

BKN Defensive eFG% when opponent on B2B in 2023-24: 53.5%

C:\Users\ssrah\AppData\Local\Temp\ipykernel_19132\2640923918.py:6: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

bkn_def['gamedate'] = pd.to_datetime(bkn_def['gamedate'])
C:\Users\ssrah\AppData\Local\Temp\ipykernel_19132\2640923918.py:21: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

bkn_def['opponent_b2b'] = b2b_mask

ANSWER 5:

- BKN Defensive eFG%: 54.3%
- When opponent on a B2B: 53.5%

Part 2 -- Trends and Visualizations

This is an intentionally open ended section, and there are multiple approaches you could take to have a successful project. Feel free to be creative. However, for this section, please consider only the density of games and travel schedule, not the relative on-court strength of different teams.

Question 6

QUESTION: Please identify at least 2 trends in scheduling over time. In other words, how are the more recent schedules different from the schedules of the past? Please include a visual (plot or styled table) highlighting or explaining each trend and include a brief written description of your findings.

ANSWER 6:

```
In [17]: import numpy as np

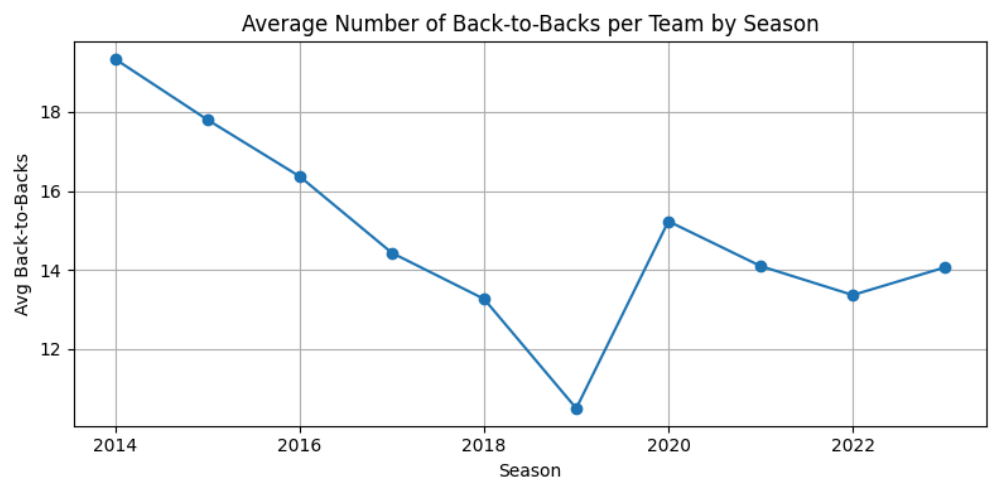
import matplotlib.pyplot as plt
```



```
# Trend 1: Fewer Back-to-Backs Over Time
schedule['gamedate'] = pd.to_datetime(schedule['gamedate'])
b2b_counts = []
for (team, season), group in schedule.groupby(['team', 'season']):
    group = group.sort_values('gamedate')
    b2b = (group['gamedate'].diff().dt.days == 1).sum()
    b2b_counts.append({'team': team, 'season': season, 'b2b': b2b})
b2b_df = pd.DataFrame(b2b_counts)
b2b_avg = b2b_df.groupby('season')['b2b'].mean()

plt.figure(figsize=(8,4))
plt.plot(b2b_avg.index, b2b_avg.values, marker='o')
plt.title('Average Number of Back-to-Backs per Team by Season')
plt.xlabel('Season')
plt.ylabel('Avg Back-to-Backs')
plt.grid(True)
plt.tight_layout()
plt.show()

# Brief written findings
print("Trend 1: The average number of back-to-back games per team has generally
COVID-19 pandemic causing schedule disruptions, which lead into the following s")
```



Trend 1: The average number of back-to-back games per team has generally decreased over time, indicating the NBA's effort to reduce schedule density and improve player rest. The notable drop in the 2019-20 season is likely due to the COVID-19 pandemic causing schedule disruptions, which lead into the following season's sudden spike back up since it was also a reduced year in terms of start and end dates.

```
In [18]: # Trend 2: Average Travel Distance Per Team Per Season
def haversine(lat1, lon1, lat2, lon2):
    R = 3959 # miles
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))

loc_map = locations.set_index('team')[['latitude', 'longitude']].to_dict('index')
travel_stats = []
for (team, season), group in schedule.groupby(['team', 'season']):
    group = group.sort_values('gamedate')
    # If home == 1, use team, else use opponent
    cities = np.where(group['home'] == 1, team, group['opponent'])
    coords = [loc_map.get(t, {'latitude': None, 'longitude': None}) for t in cities]
    lats = [c['latitude'] for c in coords]
    lons = [c['longitude'] for c in coords]
```



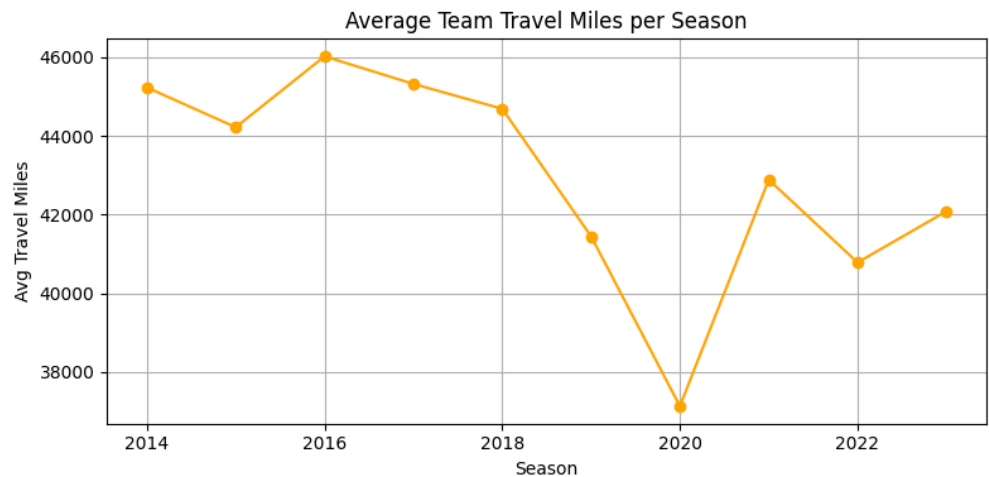
```

dist = sum(haversine(lats[i], lons[i], lats[i+1], lons[i+1])) for i in range(
travel_stats.append({'team': team, 'season': season, 'total_miles': dist})
travel_df = pd.DataFrame(travel_stats)
travel_avg = travel_df.groupby('season')['total_miles'].mean()

plt.figure(figsize=(8,4))
plt.plot(travel_avg.index, travel_avg.values, marker='o', color='orange')
plt.title('Average Team Travel Miles per Season')
plt.xlabel('Season')
plt.ylabel('Avg Travel Miles')
plt.grid(True)
plt.tight_layout()
plt.show()

# Brief written findings
print("Trend 2: The average travel distance per team per season has remained rel
" drop in the 2020-21 season is likely due to the COVID-19 pandemic causing rest

```



Trend 2: The average travel distance per team per season has remained relatively stable, but some recent seasons show a slight decrease, possibly due to more regional scheduling and reduced cross-country trips. Again, the notable drop in the 2020-21 season is likely due to the COVID-19 pandemic causing restrictions and possible stigmas against excessive traveling at the time, as well as a reduced number of games for the season.

Question 7

QUESTION: Please design a plotting tool to help visualize a team's schedule for a season. The plot should cover the whole season and should help the viewer contextualize and understand a team's schedule, potentially highlighting periods of excessive travel, dense blocks of games, or other schedule anomalies. If you can, making the plots interactive (for example through the plotly package) is a bonus.

Please use this tool to plot OKC and DEN's provided 80-game 2024-25 schedules.

ANSWER 7:

In [19]: `%pip install plotly`



Requirement already satisfied: plotly in c:\users\ssrah\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (6.3.1)

Requirement already satisfied: narwhals>=1.15.1 in c:\users\ssrah\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from plotly) (2.7.0)

Requirement already satisfied: packaging in c:\users\ssrah\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from plotly) (25.0)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.0 -> 25.2

[notice] To update, run: C:\Users\ssrah\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

```
In [20]: import plotly.graph_objects as go

def plot_team_schedule_lines(schedule_df, locations_df, team, season):
    # Filter for team and season
    df = schedule_df[(schedule_df['team'] == team) & (schedule_df['season'] == season)]
    df = df.sort_values('gamedate').reset_index(drop=True)
    df['gamedate'] = pd.to_datetime(df['gamedate'])

    # Get city coordinates for each game
    loc_map = locations_df.set_index('team')[['latitude', 'longitude']].to_dict()
    cities = np.where(df['home'] == 1, team, df['opponent'])
    coords = [loc_map.get(t, {'latitude': None, 'longitude': None}) for t in cities]
    df['latitude'] = [c['latitude'] for c in coords]
    df['longitude'] = [c['longitude'] for c in coords]

    # Calculate travel distance between games
    def haversine(lat1, lon1, lat2, lon2):
        R = 3959 # miles
        lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
        dlat = lat2 - lat1
        dlon = lon2 - lon1
        a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
        return 2 * R * np.arcsin(np.sqrt(a))

    travel = [0]
    for i in range(1, len(df)):
        if None not in (df.loc[i-1, 'latitude'], df.loc[i-1, 'longitude'], df.loc[i, 'latitude'], df.loc[i, 'longitude']):
            dist = haversine(df.loc[i-1, 'latitude'], df.loc[i-1, 'longitude'], df.loc[i, 'latitude'], df.loc[i, 'longitude'])
        else:
            dist = 0
        travel.append(dist)
    df['travel_miles'] = travel

    # Calculate games in last 6 days (density)
    density = []
    for i in range(len(df)):
        window_start = df.loc[i, 'gamedate'] - pd.Timedelta(days=5)
        games_in_window = ((df['gamedate'] >= window_start) & (df['gamedate'] <= df.loc[i, 'gamedate'])).sum()
        density.append(games_in_window)
    df['games_in_6days'] = density

    # Plot with filled area under lines and dual y-axes
    fig = go.Figure()

    # Game Density (left y-axis)
    fig.add_trace(go.Scatter(
        x=df['gamedate'], y=df['games_in_6days'],
        mode='lines+markers',
        name='Game Density (last 6 days)',
        line=dict(color='blue', width=2),
        marker=dict(size=7),
        fill='tozeroy',
```



```
fillcolor='rgba(0, 100, 255, 0.2)',
yaxis='y1'
))

# Travel Miles (right y-axis)
fig.add_trace(go.Scatter(
    x=df['gamedate'], y=df['travel_miles'],
    mode='lines+markers',
    name='Travel Miles',
    line=dict(color='orange', width=2),
    marker=dict(size=7),
    fill='tozeroy',
    fillcolor='rgba(255, 165, 0, 0.2)',
    yaxis='y2'
))

fig.update_layout(
    title=f"{team} {season} Schedule: Game Density & Travel Miles",
    xaxis_title="Date",
    yaxis=dict(
        title="Games in Last 6 Days",
        side='left',
        showgrid=False,
        zeroline=False
    ),
    yaxis2=dict(
        title="Travel Miles",
        side='right',
        overlaying='y',
        showgrid=False,
        zeroline=False
    ),
    legend=dict(x=0.75, y=1.15, orientation='h'),
    height=500,
    width=950
)
fig.show()

# Plot OKC and DEN 2024-25 schedules
plot_team_schedule_lines(draft_schedule, locations, 'OKC', 2024)
plot_team_schedule_lines(draft_schedule, locations, 'DEN', 2024)
```

Question 8

QUESTION: Using your tool, what is the best and worst part of OKC's 2024-25 draft schedule? Please give your answer as a short brief to members of the front office and coaching staff to set expectations going into the season. You can include context from past schedules.

```
In [21]: # Get OKC's average number of 4-in-6 stretches per season (per 82 games) from re
okc_stretches = results_df[results_df['team'] == 'OKC'][['season', 'stretches_per
print(okc_stretches)
```



	season	stretches_per_82
200	2014	34.000000
201	2015	26.000000
202	2016	26.000000
203	2017	18.000000
204	2018	18.000000
205	2019	23.916667
206	2020	37.583333
207	2021	25.000000
208	2022	26.000000
209	2023	27.000000

```
In [22]: # Ensure gamedate is datetime
schedule['gamedate'] = pd.to_datetime(schedule['gamedate'])

# Get OKC games from schedule
okc_hist = schedule[schedule['team'] == 'OKC'].copy()
okc_hist = okc_hist.sort_values('gamedate').reset_index(drop=True)

# Get Location mapping
loc_map = locations.set_index('team')[['latitude', 'longitude']].to_dict('index')

def haversine(lat1, lon1, lat2, lon2):
    R = 3959 # miles
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))

# Calculate total travel miles for each season
okc_travel = []
for season, group in okc_hist.groupby('season'):
    group = group.sort_values('gamedate')
    cities = np.where(group['home'] == 1, 'OKC', group['opponent'])
    coords = [loc_map.get(t, {'latitude': None, 'longitude': None}) for t in cities]
    lats = [c['latitude'] for c in coords]
    lons = [c['longitude'] for c in coords]
    dist = sum(haversine(lats[i], lons[i], lats[i+1], lons[i+1]) for i in range(len(lats)-1))
    okc_travel.append({'season': season, 'total_miles': dist})

okc_travel_df = pd.DataFrame(okc_travel)
print(okc_travel_df)
```

	season	total_miles
0	2014	44155.501730
1	2015	48217.985540
2	2016	50610.585893
3	2017	48825.069501
4	2018	45943.378218
5	2019	41992.960367
6	2020	32276.638302
7	2021	43099.302713
8	2022	43428.385573
9	2023	40170.903785

```
In [27]: # Calculate total miles traveled by OKC in 2024-25 draft schedule
okc_2024_games = okc_games.sort_values('gamedate').reset_index(drop=True)
cities = np.where(okc_2024_games['home'] == 1, 'OKC', okc_2024_games['opponent'])
lats = [loc_map[t]['latitude'] for t in cities]
lons = [loc_map[t]['longitude'] for t in cities]

def haversine(lat1, lon1, lat2, lon2):
    R = 3959 # miles
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
```



```

dlon = lon2 - lon1
a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
return 2 * R * np.arcsin(np.sqrt(a))

total_miles_2024 = sum(
    haversine(lats[i], lons[i], lats[i+1], lons[i+1])
    for i in range(len(lats)-1)
)

print(f"OKC total miles traveled in 2024-25 draft schedule: {total_miles_2024:.6}")
print(f"OKC 4-in-6 stretches in 2024-25 draft schedule: {count_4_in_6}")

```

OKC total miles traveled in 2024-25 draft schedule: 41451
 OKC 4-in-6 stretches in 2024-25 draft schedule: 26

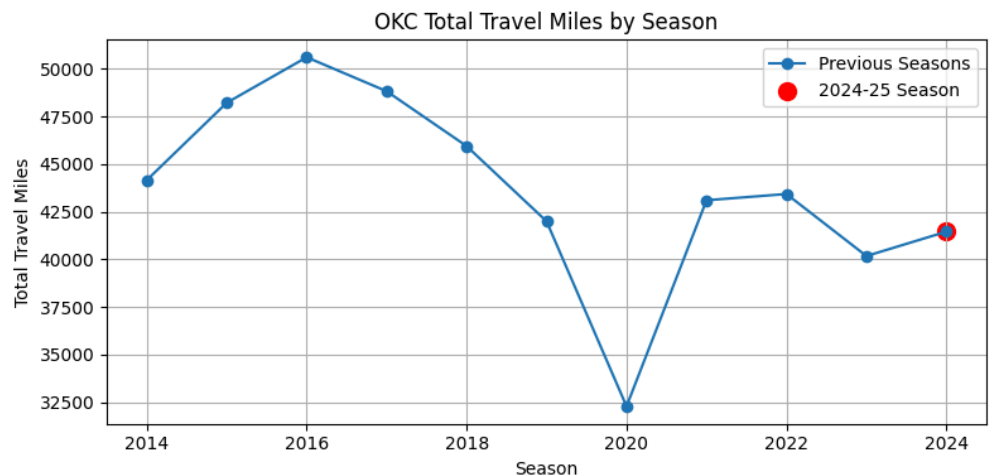
```

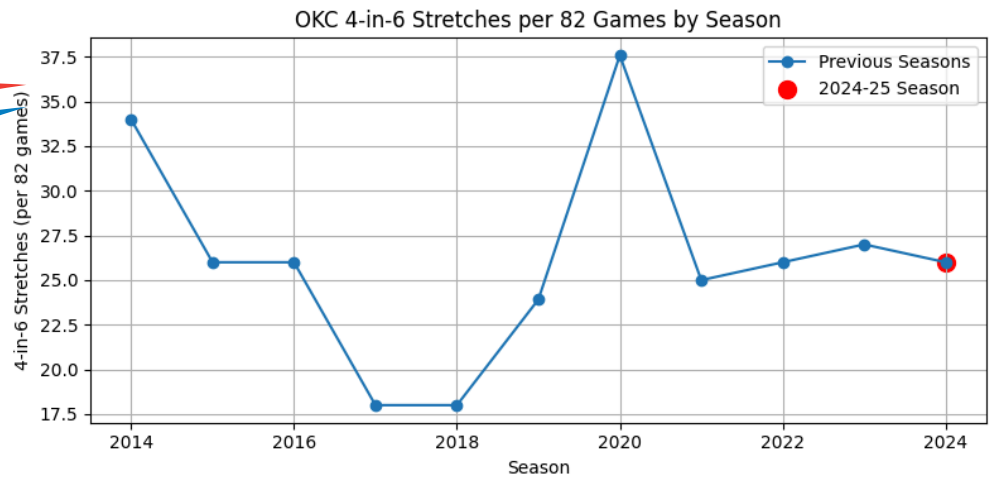
In [32]: import matplotlib.pyplot as plt
# Append 2024-25 values to historical datasets
okc_travel_df.loc[len(okc_travel_df)] = [2024, total_miles_2024]
okc_stretches.loc[len(okc_stretches)] = [2024, count_4_in_6]

# Plot OKC travel miles by season, highlight 2024-25
plt.figure(figsize=(8,4))
plt.plot(okc_travel_df['season'], okc_travel_df['total_miles'], marker='o', label='Previous Seasons')
plt.scatter(2024, total_miles_2024, color='red', s=100, label='2024-25 Season')
plt.title("OKC Total Travel Miles by Season")
plt.xlabel("Season")
plt.ylabel("Total Travel Miles")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Plot OKC 4-in-6 stretches by season, highlight 2024-25
plt.figure(figsize=(8,4))
plt.plot(okc_stretches['season'], okc_stretches['stretches_per_82'], marker='o', label='Previous Seasons')
plt.scatter(2024, count_4_in_6, color='red', s=100, label='2024-25 Season')
plt.title("OKC 4-in-6 Stretches per 82 Games by Season")
plt.xlabel("Season")
plt.ylabel("4-in-6 Stretches (per 82 games)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```





ANSWER 8:

Generally speaking, compared to the previous 3 years, both the game density and total travel milage remain similar heading into the 2024-25 season.

However, breaking the season down, we can see the following:

- The least game-packed parts of the season seem to come between late November to mid/late December, although those include some periods of longer travels.
- Some of the more dense parts of the season come in early through mid November, most of January, and from late February into mid-March. However, there are longer periods of little to no travel for those November and January blocks.
- February and early March show some spikes in traveling, but late March into early April is a bit more calm with less travel and no 4-in-6 stretches for a little while until the very end of the regular season in mid-April.

Part 3 -- Modeling

Question 9

QUESTION: Please estimate how many more/fewer regular season wins each team has had due to schedule-related factors from 2019-20 though 2023-24. Your final answer should have one number for each team, representing the total number of wins (not per 82, and not a per-season average). You may consider the on-court strength of the scheduled opponents as well as the impact of travel/schedule density. Please include the teams and estimates for the most helped and most hurt in the answer key.

If you fit a model to help answer this question, please write a paragraph explaining your model, and include a simple model diagnostic (eg a printed summary of a regression, a variable importance plot, etc).

```
In [ ]: from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
import numpy as np

import matplotlib.pyplot as plt

# Merge opponent win% from game_data
opp_win_pct = (
```



```

game_data.groupby(['off_team', 'season'])['off_win']
    .mean()
    .reset_index()
    .rename(columns={'off_team': 'opponent', 'off_win': 'opp_win_pct'})
)
sched = schedule[schedule['season'].isin([2019, 2020, 2021, 2022, 2023]).copy()]
sched = sched.merge(opp_win_pct, on=['opponent', 'season'], how='left')

# Add travel miles feature
loc_map = locations.set_index('team')[['latitude', 'longitude']].to_dict('index')
sched = sched.sort_values(['team', 'gamedate'])
sched['latitude'] = sched.apply(lambda r: loc_map[r['team']]['latitude'] if r['team'] in loc_map else None, axis=1)
sched['longitude'] = sched.apply(lambda r: loc_map[r['team']]['longitude'] if r['team'] in loc_map else None, axis=1)
sched['prev_latitude'] = sched.groupby('team')['latitude'].shift(1)
sched['prev_longitude'] = sched.groupby('team')['longitude'].shift(1)
def haversine(lat1, lon1, lat2, lon2):
    R = 3959
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
sched['travel_miles'] = sched.apply(
    lambda r: haversine(r['prev_latitude'], r['prev_longitude'], r['latitude'],
    if pd.notnull(r['prev_latitude']) else 0, axis=1
)

# Add back-to-back feature
sched = sched.sort_values(['team', 'gamedate'])
sched['gamedate'] = pd.to_datetime(sched['gamedate'])
sched['prev_gamedate'] = sched.groupby('team')['gamedate'].shift(1)
sched['back_to_back'] = (sched['gamedate'] - sched['prev_gamedate']).dt.days == 1
sched['back_to_back'] = sched['back_to_back'].astype(int)

# Add rolling opponent win% (last 10 games before each game)
game_data['gamedate'] = pd.to_datetime(game_data['gamedate'])
game_data = game_data.sort_values(['off_team', 'gamedate'])
game_data['opp_rolling_win_pct'] = (
    game_data.groupby('off_team')['off_win']
    .transform(lambda x: x.rolling(window=10, min_periods=1).mean())
)
sched = sched.merge(
    game_data[['gamedate', 'off_team', 'opp_rolling_win_pct']],
    left_on=['gamedate', 'opponent'],
    right_on=['gamedate', 'off_team'],
    how='left'
)

# Add opponent margin of victory (last game)
game_data['opp_margin'] = game_data['points'] - game_data.groupby('off_team')['points'].shift(1)
sched = sched.merge(
    game_data[['gamedate', 'off_team', 'opp_margin']],
    left_on=['gamedate', 'opponent'],
    right_on=['gamedate', 'off_team'],
    how='left'
)

# Add more candidate features
sched['home_advantage'] = sched['home']
sched['season_game_num'] = sched.groupby(['team', 'season']).cumcount() + 1
sched['is_weekend'] = sched['gamedate'].dt.weekday.isin([5, 6]).astype(int)
sched['opp_travel_miles'] = sched.groupby(['opponent', 'season'])['travel_miles'].shift(1)
sched['team_travel_miles'] = sched.groupby(['team', 'season'])['travel_miles'].shift(1)
sched['team_win_pct_season'] = sched.groupby(['team', 'season'])['win'].transform(lambda x: x.rolling(window=10, min_periods=1).mean())
sched['opp_win_pct_season'] = sched.groupby(['opponent', 'season'])['win'].transform(lambda x: x.rolling(window=10, min_periods=1).mean())
sched['team_last10_win_pct'] = sched.groupby('team')['win'].transform(lambda x: x.rolling(window=10, min_periods=1).mean())

```



```
sched['opp_last10_win_pct'] = sched.groupby('opponent')['win'].transform(lambda x: x - x.mean())

# Candidate features
candidate_features = [
    'back_to_back', 'travel_miles', 'opp_win_pct', 'opp_rolling_win_pct', 'opp_rank',
    'home_advantage', 'season_game_num', 'is_weekend', 'opp_travel_miles', 'team_rank',
    'team_win_pct_season', 'opp_win_pct_season', 'team_last10_win_pct', 'opp_last10_win_pct'
]

# Drop rows with missing values in candidate features
sched = sched.dropna(subset=candidate_features + ['win'])

# Feature selection using Random Forest importance
X_full = sched[candidate_features]
y_full = sched['win']
rf_full = RandomForestRegressor(n_estimators=100, random_state=42)
rf_full.fit(X_full, y_full)
importances_full = rf_full.feature_importances_

# Select top N features by importance
N = 7
top_features = [f for _, f in sorted(zip(importances_full, candidate_features), reverse=True)]

print("Top features by RF importance:", top_features)

# Prepare features and target for regression
X = sched[top_features]
y = sched['win']

# Fit regression model
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)
print("Regression R^2:", r2_score(y, y_pred))
print("Coefficients:", dict(zip(top_features, model.coef_)))

# Variable importance with Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X, y)
importances = rf.feature_importances_
plt.figure(figsize=(8, 5))
plt.bar(top_features, importances)
plt.title("Feature Importances (Random Forest)")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()

# Estimate schedule impact: predict win probability with actual vs. "neutral" schedule
sched['pred_actual'] = model.predict(X)
X_neutral = X.copy()
for f in top_features:
    X_neutral[f] = X[f].mean()
sched['pred_neutral'] = model.predict(X_neutral)
sched['schedule_win_diff'] = sched['pred_actual'] - sched['pred_neutral']

# Aggregate by team
team_schedule_impact = sched.groupby('team')['schedule_win_diff'].sum().sort_values()
team_schedule_impact_df = team_schedule_impact.reset_index()
team_schedule_impact_df.columns = ['team', 'schedule_win_diff']
display(team_schedule_impact_df)
```

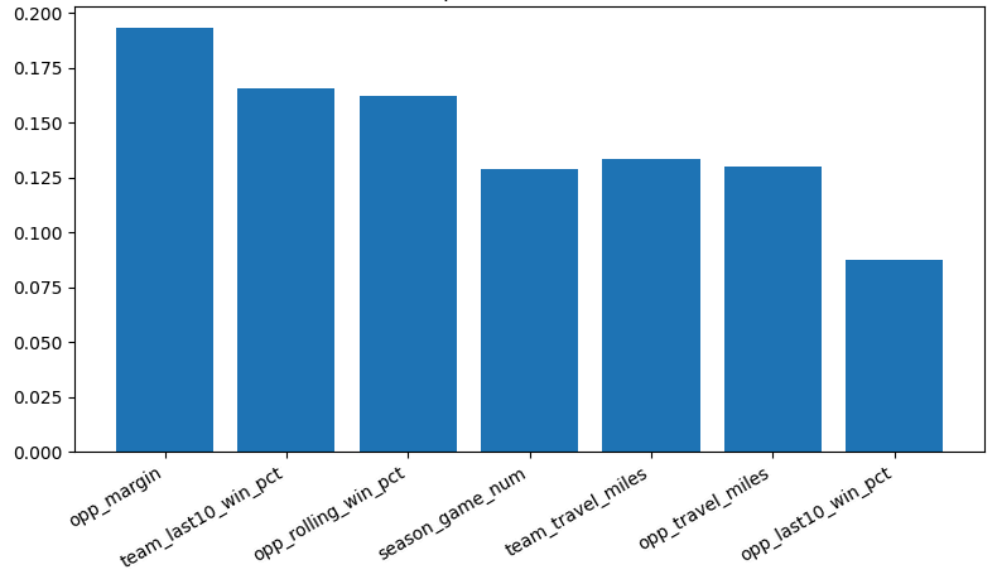


Top features by RF importance: ['opp_margin', 'team_last10_win_pct', 'opp_rolling_win_pct', 'season_game_num', 'team_travel_miles', 'opp_travel_miles', 'opp_last10_win_pct']

Regression R²: 0.38847806203142576

Coefficients: {'opp_margin': np.float64(-0.008125925875787412), 'team_last10_win_pct': np.float64(0.657861171306425), 'opp_rolling_win_pct': np.float64(-0.7525287329650363), 'season_game_num': np.float64(5.0424326068110314e-05), 'team_travel_miles': np.float64(-2.8575026391641778e-05), 'opp_travel_miles': np.float64(1.5146904385511528e-06), 'opp_last10_win_pct': np.float64(0.4314881234478678)}

Feature Importances (Random Forest)





	team	schedule_win_diff
0	MIL	74.209344
1	BOS	70.972302
2	DEN	68.213405
3	PHX	62.073600
4	PHI	58.817068
5	LAC	48.329059
6	MIA	47.193607
7	DAL	40.256519
8	LAL	28.821119
9	UTA	23.039103
10	MEM	17.857390
11	BKN	12.581285
12	NYK	8.648059
13	TOR	-1.042112
14	CLE	-2.189012
15	MIN	-4.775709
16	OKC	-13.420714
17	GSW	-14.637750
18	CHI	-16.642004
19	NOP	-18.125618
20	IND	-28.670642
21	SAC	-29.688720
22	ATL	-32.613730
23	POR	-35.723617
24	ORL	-36.150554
25	CHA	-37.452443
26	WAS	-60.921186
27	SAS	-65.376081
28	HOU	-67.193257
29	DET	-96.388709

ANSWER 9:

- Most Helped by Schedule: MIL (74.2 wins)
- Most Hurt by Schedule: DET (-96.3 wins)
- As a caveat: The model would end up considering more features that are focused on the on-court variables that determine the strength of opponents for a given game if



we continue to add features. But, to limit the focus to the kind of features that pertain to scheduling specifically, things like travel, game density (back-to-backs in this case), overall strength of both a team and their opponent based on their rolling win percentage, how far into the season the game is, etc are most pertinent to consider in this model scenario. This can explain why we end up going with a model with a somewhat lower R-squared value - roughly 39% of the time can the win differential be explained by this model. However, continuing to add more features would just end up ballooning the total win differentials to the point where we aren't explaining it based mostly on the factors of team schedules, but just just predicting win differential as a whole.